



DBD-281 PROJECT

Jason Crous (601662)
Kamohelo Mabena (602556)
Mufunwa Muofhe (602679)
Mkhanyisi Mqadi (603000)

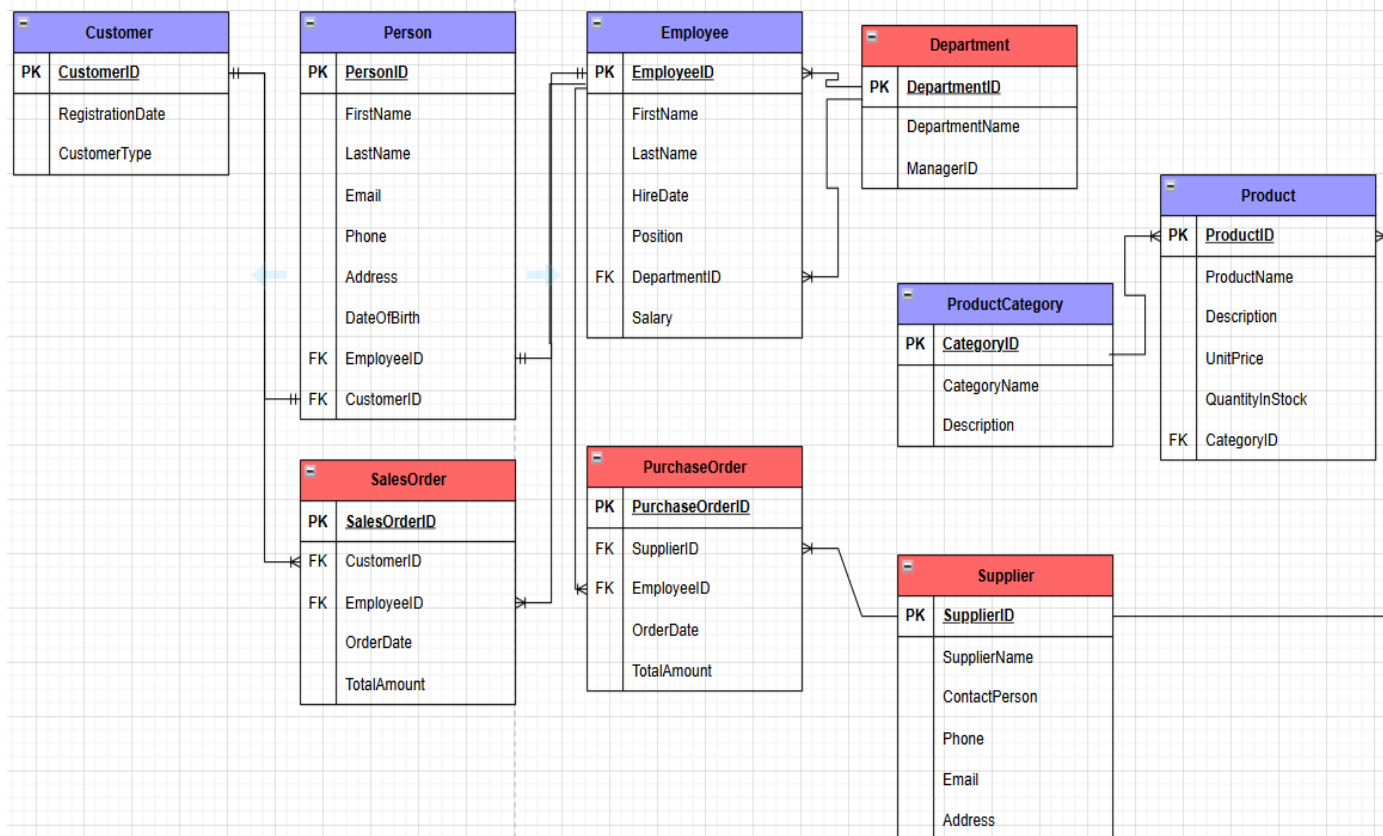


APRIL 19, 2025
ADVENTURE WORKING
999 Spooner Street

Introduction/Background:

Adventure Working is a dynamic and multifaceted company that operates across various core business functions, including production, human resources, purchasing, and sales. With a strong emphasis on operational efficiency and data-driven decision-making, the company maintains comprehensive records across all departments.

Its well-structured database captures detailed information about personnel, supply chains, product output, and customer transactions, reflecting an integrated approach to enterprise management. Adventure Working's commitment to maintaining accurate and organised data supports its strategic goals, enabling the company to respond effectively to internal needs and market demands.



Normalisation Steps Taken:

1NF (First Normal Form) – Atomicity

To meet **1NF**, we ensured:

- **All attributes hold atomic (indivisible) values.**
- No repeating groups or arrays.

Examples:

- Address is not split into multiple lines (e.g., Street, City, ZIP) in this version for simplicity, but can be if required.
- Phone and Email are stored as individual fields, not comma-separated lists.

2NF (Second Normal Form) – Eliminate Partial Dependencies

To meet **2NF**, we ensured:

- The database is already in 1NF.
- **Every non-key attribute fully functionally depends on the entire primary key**, especially for tables with composite primary keys (like junction tables).

Examples:

- In **SalesOrder (SalesOrderID, ProductID)**, attributes like Quantity and UnitPrice depend on **both** SalesOrderID and ProductID, not just one.
- The **Employee table** has attributes like HireDate and Position that depend only on EmployeeID, not a partial key.

3NF (Third Normal Form) – Eliminate Transitive Dependencies

To meet **3NF**, we ensured:

- The database is in 2NF.
- **No non-prime attribute depends on another non-prime attribute** (i.e., no transitive dependencies).

Examples:

- In the **Product** table, CategoryID is a foreign key to the **ProductCategory** table. This removes transitive dependency on CategoryName and Description.
- The **Department** table stores the DepartmentName and links to a ManagerID, a foreign key to **Employee**, keeping each piece of information in its logical place.

Justification for 3NF:

The database design satisfies **Third Normal Form** because:

- **All attributes in every table directly depend on the primary key** and nothing else.
- **No duplicate data is stored** across unrelated entities. For instance:
 - Supplier details are only stored in the **Supplier** table.
 - Product categories are isolated in a **Category** table rather than being repeated in the **Product** table.

Logical Design (Data Dictionary & Object Descriptions)

AdventureWorks Data Dictionary:

Table Name	Column Name	Data Type	PK/FK	Constraints	Description
Person.Person	BusinessEntityID	INT	PK	NOT NULL, Identity	Unique identifier for the person.
Person.Person	FirstName	NVARCHAR(50)		NOT NULL	First name of the individual.
Person.Person	LastName	NVARCHAR(50)		NOT NULL	Last name of the individual.
Person.Person	ModifiedDate	DATETIME		NOT NULL	Last date the record was updated.
Sales.Customer	CustomerID	INT	PK	NOT NULL, Identity	Unique identifier for each customer.
Sales.Customer	PersonID	INT	FK	NULLABLE	Links to the Person table if individual customer.
Sales.Customer	StoreID	INT	FK	NULLABLE	Links to a store.
Sales.Customer	TerritoryID	INT	FK		Links to the sales territory.
Sales.SalesOrderHeader	SalesOrderID	INT	PK	NOT NULL, Identity	Main identifier for a sales order.
Sales.SalesOrderHeader	CustomerID	INT	FK		Identifies the customer placing the order.
Sales.SalesOrderHeader	OrderDate	DATETIME		NOT NULL	The date the order was placed.
Sales.SalesOrderDetail	SalesOrderDetailID	INT	PK	NOT NULL, Identity	Line item identifier within a sales order.
Sales.SalesOrderDetail	SalesOrderID	INT	FK		The sales order to which this detail belongs.
Sales.SalesOrderDetail	ProductID	INT	FK		ID of the product being sold.
Sales.SalesOrderDetail	UnitPrice	MONEY		NOT NULL	Price per unit of product.
Production.Product	ProductID	INT	PK	NOT NULL, Identity	Identifier for each product.
Production.Product	Name	NVARCHAR(50)		NOT NULL	Name of the product.
Production.Product	ProductNumber	NVARCHAR(25)		NOT NULL	Unique product code or SKU.

Object Descriptions:

Object Descriptions		
Object Type	Name	Description
Table	Person.Person	Stores personal details like name, contact info, and title.
Table	Sales.SalesOrderHeader	Represents customer sales orders.
Table	Production.Product	Contains data about items sold by the company.
View	Top10BestSoldProducts	Displays top 10 best-selling products based on total quantity sold, and how many distinct orders did each appear in
Trigger		Automatically updates the ModifiedDate when a record is changed.
Procedure		Returns a list of orders for a specified customer ID.

Proposed Database Environment:

Database Platform:

SQL Server Management Studio 2022

Our reasoning for choosing this platform is that SQL Server Management Studio 2022 supports enterprise-grade features like stored procedures, triggers, role-based security, and performance optimisation tools.

Database Structure Overview:

Database Name: AdventureWorks2022

Includes tables for:

- Person, Employee, Customer
- Product, ProductCategory, Supplier
- SalesOrder, SalesOrderDetail, PurchaseOrder, PurchaseOrderDetail
- Department, UserLogins, and logs such as ProductLog

Also Includes:

- **Views** for reporting
- **Stored Procedures** for dynamic access
- **Triggers** for change tracking
- **Functions** for business logic
- **Indexes** for performance

Data File Organisation:

Primary Filegroup (PRIMARY)

- Contains core schema and primary data tables.

Secondary Filegroups (optional, for performance)

- **INDEXES:** Store non-clustered indexes for large transactional tables (like SalesOrderDetail, PurchaseOrderDetail)
- **ARCHIVE:** Older order and log data can be moved here for storage efficiency.

File Types:

- **.MDF** – Primary data file.
- **.NDF** – Optional secondary files (indexes, archived data).
- **.LDF** – Transaction log file for recovery and rollback.

Security Requirements:

Authentication and Authorisation:

Mixed-mode authentication: (SQL & Windows).

Role-Based Access Control (RBAC)

- **SalesRole:** SELECT/INSERT on sales tables/views
- **HRRole:** SELECT/UPDATE on employee records
- **AdminRole:** Full DB access
- **AuditorRole:** Read-only access for reports

User Permissions:

- Use CREATE LOGIN and CREATE USER to grant least-privilege access.
- Sensitive data (e.g., salaries, contact info) restricted by views with filtering or column masking.

Audit & Monitoring:

- ProductLog trigger captures stock changes.
- Use SQL Server Management Studio Audit (or PostgreSQL logging) to track access/modifications.
- Scheduled reports on changes to critical tables.

Performance Requirements:

Indexing:

- Clustered indexes on primary keys (ProductID, SalesOrderID, etc.)
- Non-clustered indexes on frequently filtered fields (OrderDate, CustomerID, EmployeeID)
- Use covered indexes for reporting queries

Query Optimisation:

- Use parameterised stored procedures instead of ad-hoc queries.
- Regular use of execution plan analysis for heavy queries.
- Avoid table scans by ensuring correct indexing.

Data Volume Handling:

- Partition large tables by OrderDate or DepartmentID (optional).
- Archive old orders/purchases periodically to reduce load.

Maintenance:

- Set up daily backups (full + differential + transaction log).
- Regular index maintenance: REBUILD or REORGANISE.
- DBCC CHECKDB for integrity checks.

Physical Database Implementation:

The Physical Database has been sent on Teams, and the query pages are in the Project folder.